

School of Computer Science and Artificial Intelligence

Lab Assignment # 1

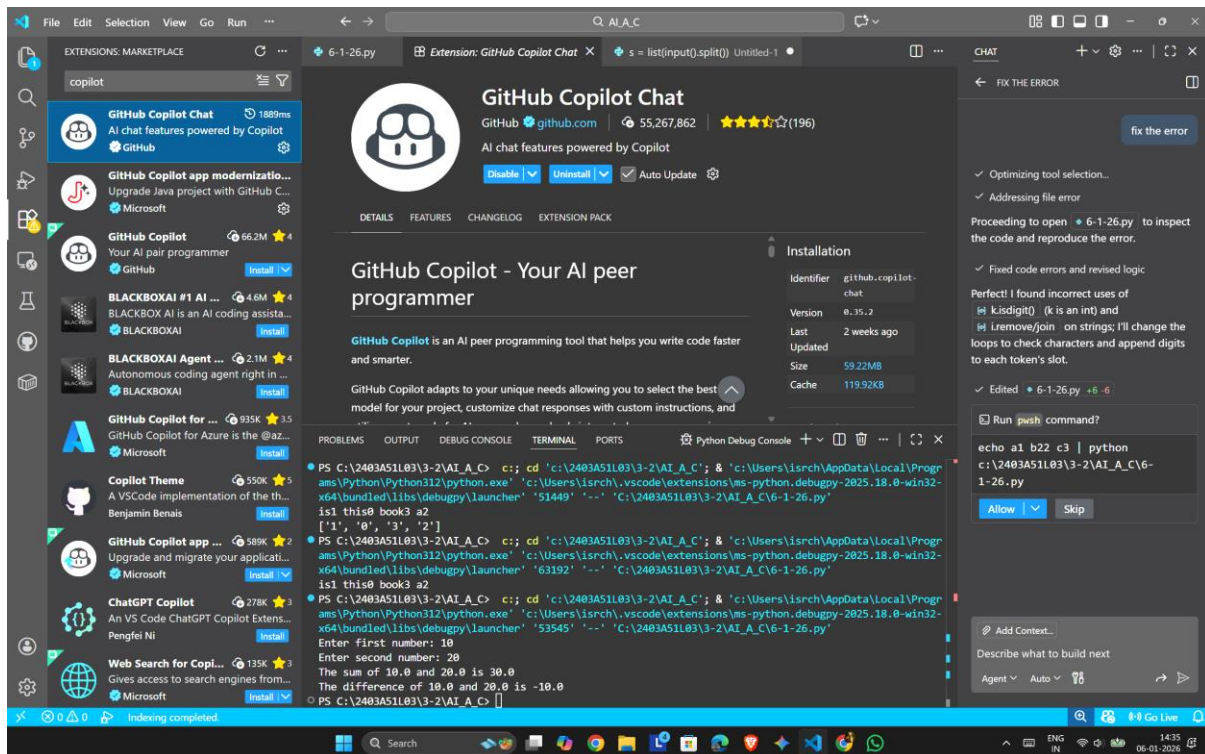
Program : B. Tech (CSE)
Specialization :
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : II
Academic Session : 2025-2026
Name of Student : S.Sai Kumar
Enrollment No. : 2403A51L46
Batch No. : 52
Date : 06/01/26

Submission Starts here

Screenshots:

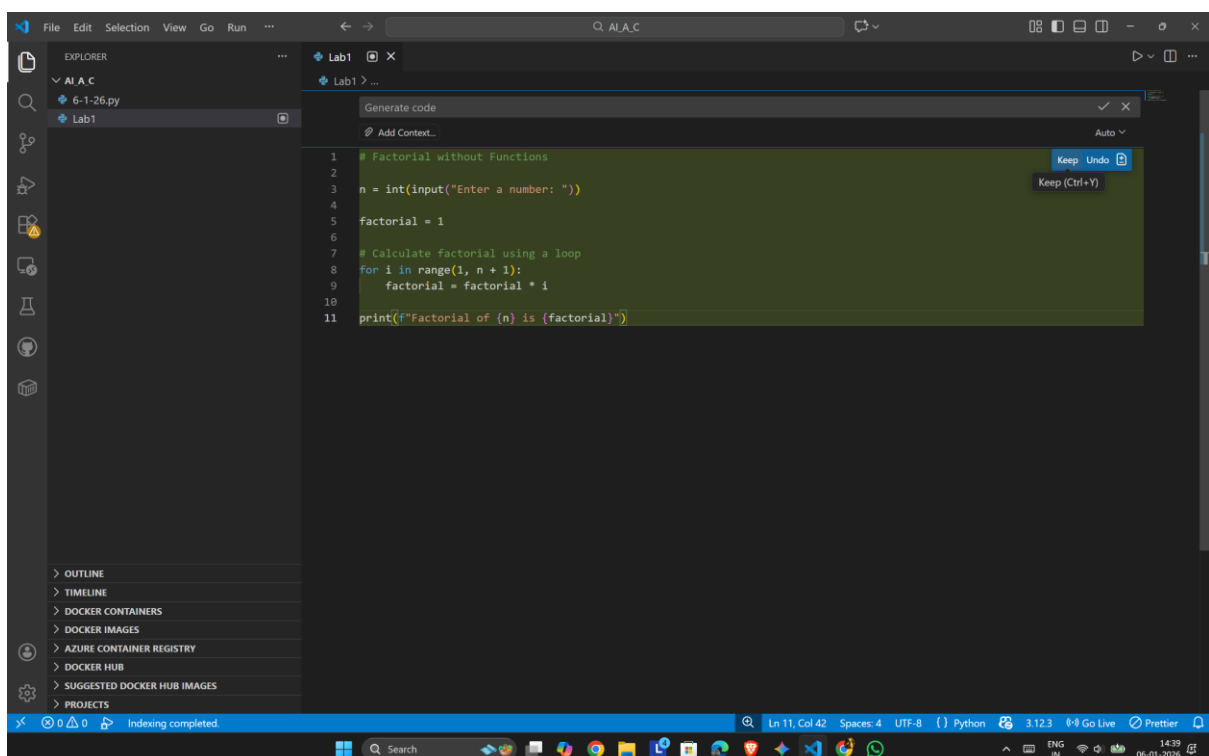
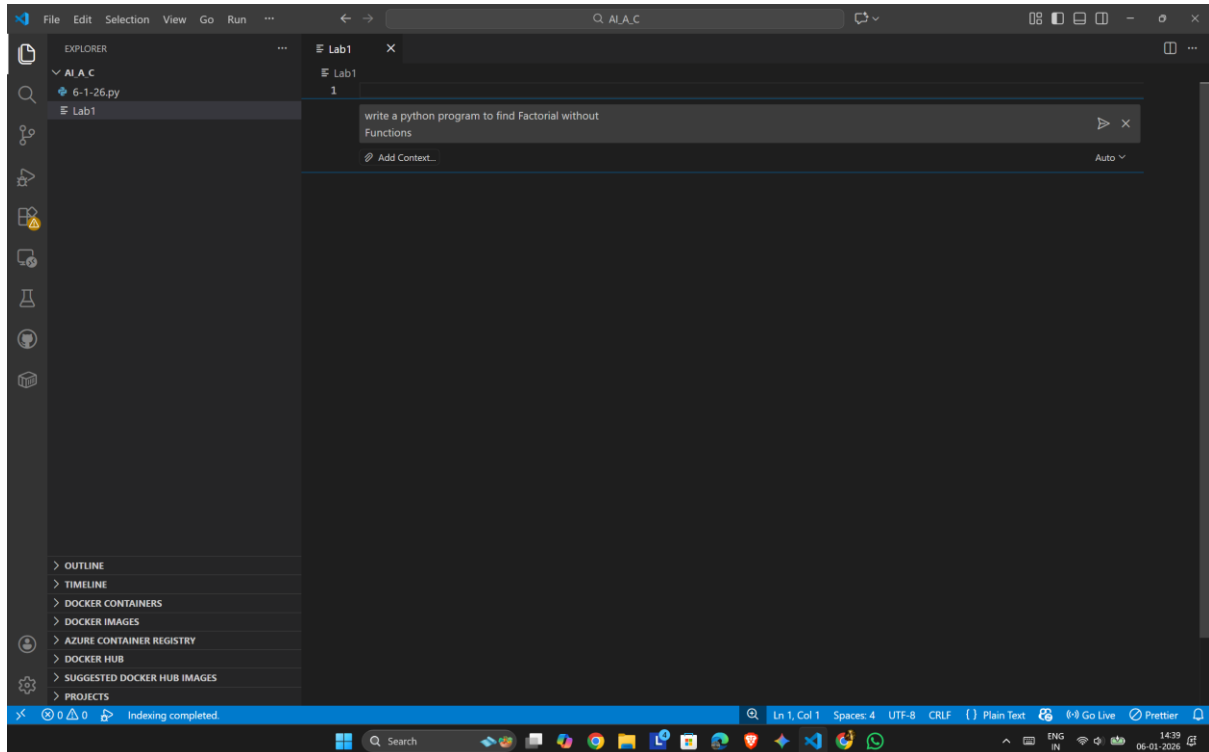
TASK - 0

- Install “Github Copilot” in Visual Studio Code

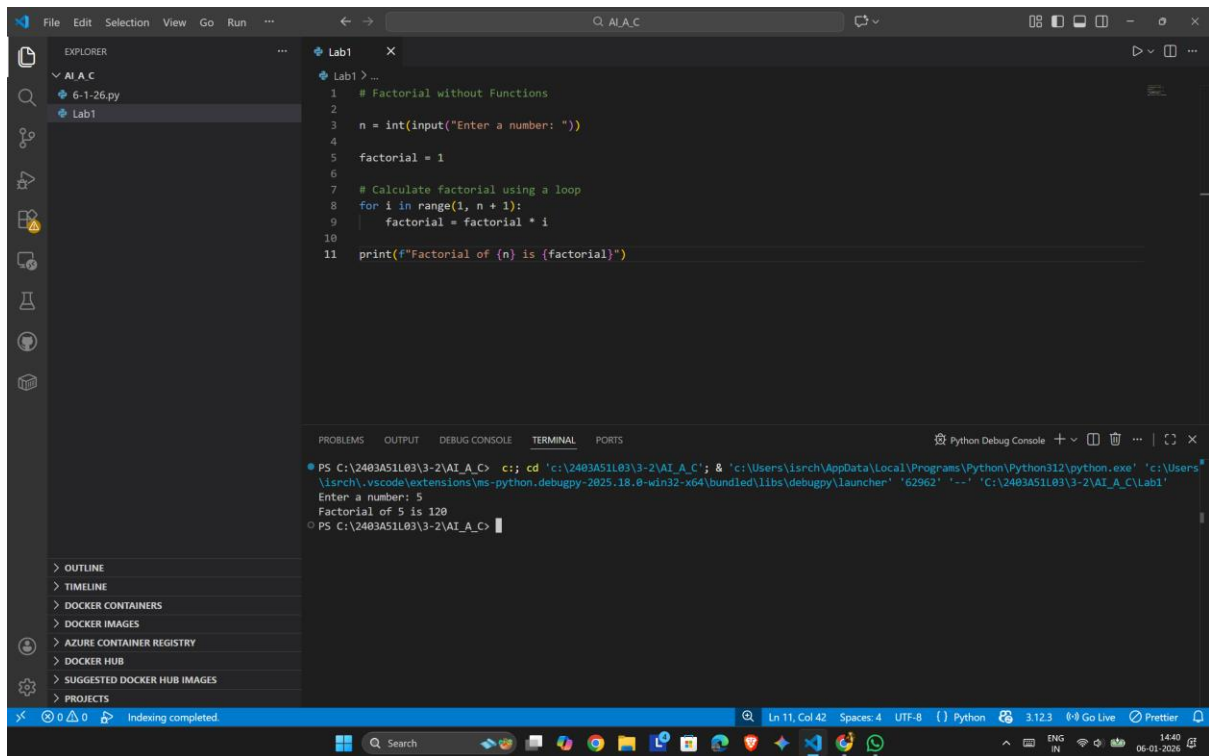


- Task Description

Use GitHub Copilot to generate a Python program that computes a mathematical product-based value (factorial-like logic) directly in the main execution flow, without using any user-defined functions.



OUTPUT:



The screenshot shows a Visual Studio Code editor window with a Python file named 'Lab1'. The code calculates the factorial of a number using a loop. The terminal output shows the program running, prompting for a number (5) and displaying the result (Factorial of 5 is 120).

```
1 # Factorial without Functions
2
3 n = int(input("Enter a number: "))
4
5 factorial = 1
6
7 # Calculate factorial using a loop
8 for i in range(1, n + 1):
9     factorial = factorial * i
10
11 print(f"Factorial of {n} is {factorial}")
```

Terminal Output:

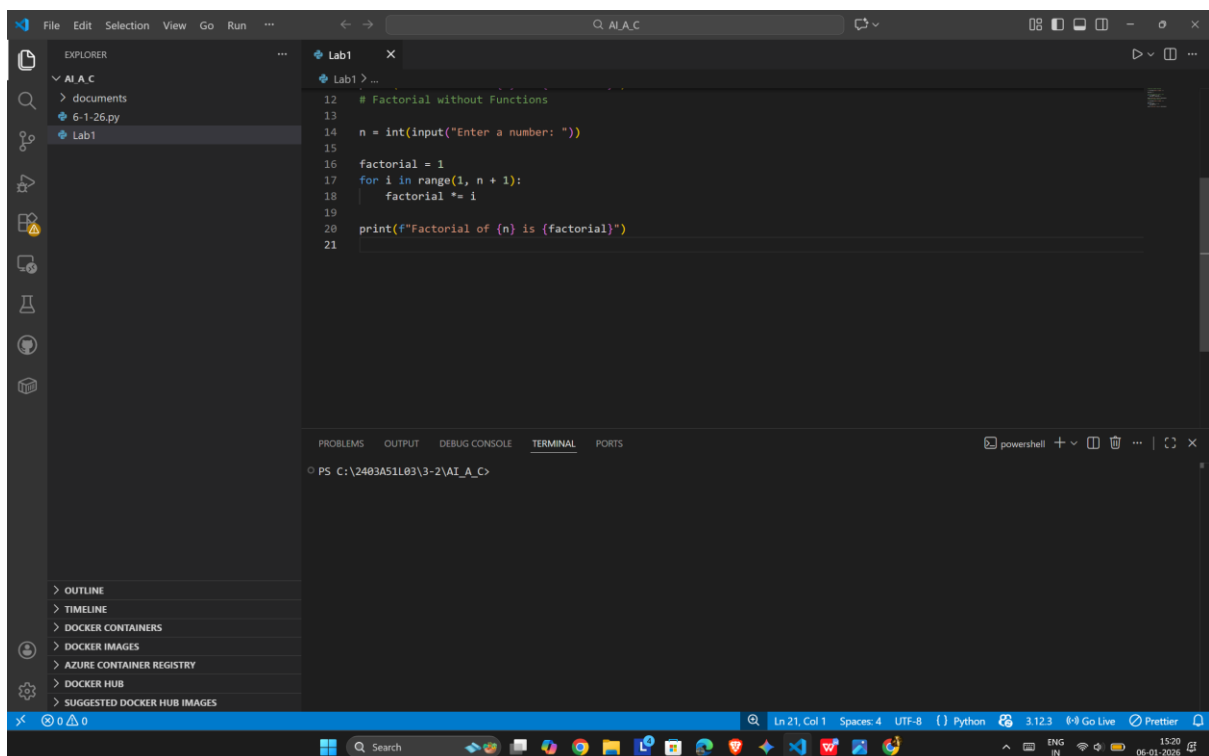
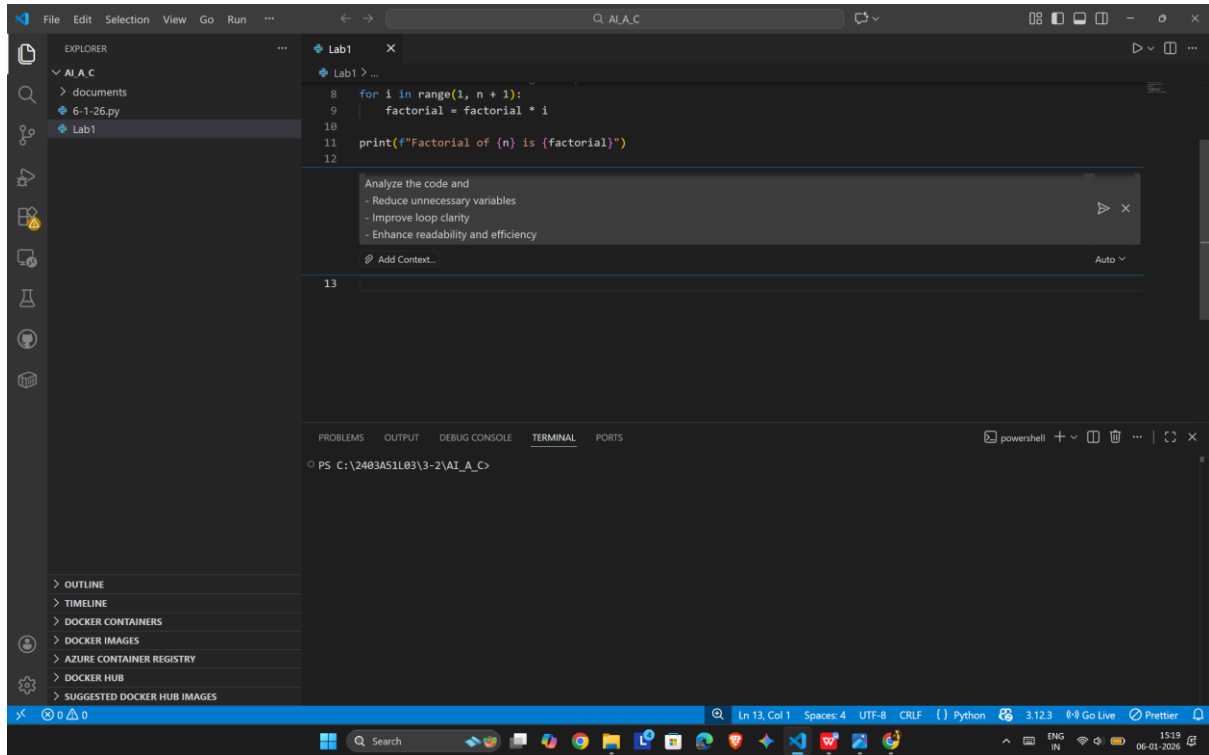
```
PS C:\2403A51L03\3-2\AI_A_C> c:\Users\isrch\AppData\Local\Programs\Python\Python312\python.exe 'c:\Users\isrch\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\lib\debugpy\launcher' '62962' '--' 'C:\2403A51L03\3-2\AI_A_C\Lab1'
Enter a number: 5
Factorial of 5 is 120
PS C:\2403A51L03\3-2\AI_A_C>
```

- The Copilot is very helpful because we can generate code by just giving a prompt in Copilot Chat (ctrl + I)
- The code generated was as requested in the prompt

Task Description

Analyze the code generated in Task 1 and use Copilot again to:

- Reduce unnecessary variables
- Improve loop clarity
- Enhance readability and efficiency



```

1
2 # Factorial without Functions
3
4 n = int(input("Enter a number: "))
5
6 factorial = 1
7 for i in range(1, n + 1):
8     factorial *= i
9
10 print(f"Factorial of {n} is {factorial}")
11
12

```

Terminal Output:

```

PS C:\2403A51L03\3-2\AI_A_C>
PS C:\2403A51L03\3-2\AI_A_C> & 'c:\Users\isrch\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\isrch\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62973' '--' 'c:\2403A51L03\3-2\AI_A_C\Lab1'
Enter a number: 6
Factorial of 6 is 720
PS C:\2403A51L03\3-2\AI_A_C> cd 'c:\2403A51L03\3-2\AI_A_C'; & 'c:\Users\isrch\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\isrch\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51942' '--' 'c:\2403A51L03\3-2\AI_A_C\Lab1'
Enter a number: 5
Factorial of 5 is 120
PS C:\2403A51L03\3-2\AI_A_C>

```

What was improved?

- Shorter multiplication statement
- $\text{factorial} = \text{factorial} * i \rightarrow \text{factorial} *= i$
- Removed unnecessary comment
- The loop logic is self-explanatory, so the comment was removed.

Why the new version is better?

1. Readability

$*$ is clearer and more concise.

- Fewer lines and less clutter make the code easier to read.

2. Maintainability

- Cleaner code is easier to modify and debug.
- Reduced redundancy lowers the chance of mistakes.

3. Performance

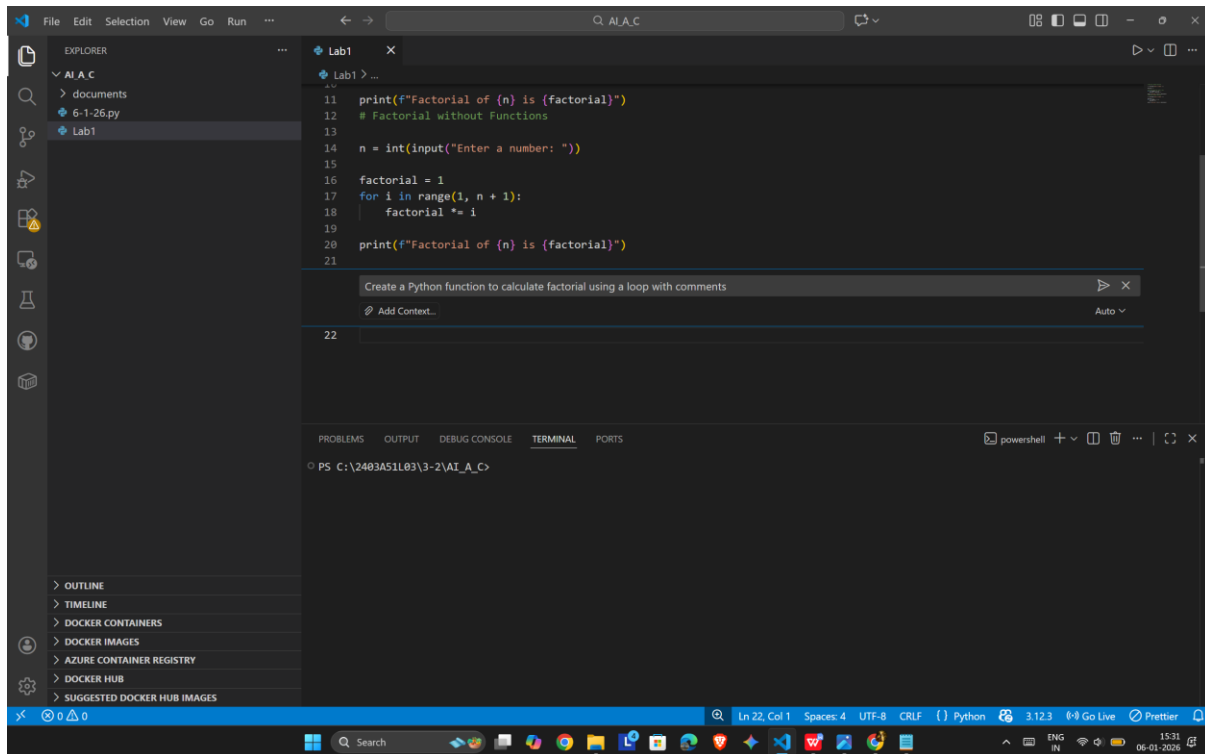
- Performance is effectively the same.

$*$ is marginally optimized at the bytecode level, but the difference is negligible.

Task Description

Use GitHub Copilot to generate a modular version of the program by:

- Creating a user-defined function
- Calling the function from the main block



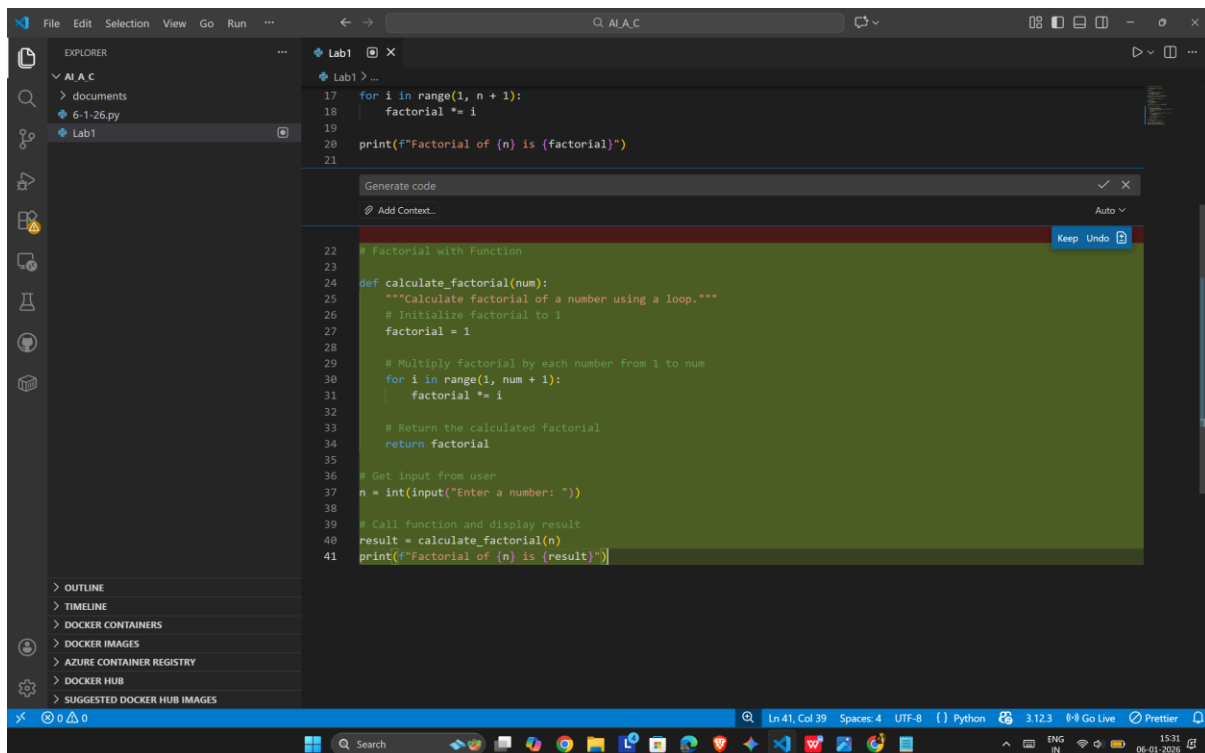
```

11 print(f"Factorial of {n} is {factorial}")
12 # Factorial without Functions
13
14 n = int(input("Enter a number: "))
15
16 factorial = 1
17 for i in range(1, n + 1):
18     factorial *= i
19
20 print(f"Factorial of {n} is {factorial}")
21

```

Create a Python function to calculate factorial using a loop with comments

Add Context... Auto



```

17 for i in range(1, n + 1):
18     factorial *= i
19
20 print(f"Factorial of {n} is {factorial}")
21

```

Generate code

Add Context... Auto

```

22 # Factorial with Function
23
24 def calculate_factorial(num):
25     """Calculate factorial of a number using a loop."""
26     # Initialize factorial to 1
27     factorial = 1
28
29     # Multiply factorial by each number from 1 to num
30     for i in range(1, num + 1):
31         factorial *= i
32
33     # Return the calculated factorial
34     return factorial
35
36 # Get input from user
37 n = int(input("Enter a number: "))
38
39 # Call function and display result
40 result = calculate_factorial(n)
41 print(f"Factorial of {n} is {result}")

```

The screenshot shows the Visual Studio Code (VS Code) interface. The main editor window displays a Python script named 'Lab1' with the following code:

```

1 # Factorial with Function
2
3
4 def calculate_factorial(num):
5     """Calculate factorial of a number using a loop."""
6     # Initialize factorial to 1
7     factorial = 1
8
9     # Multiply factorial by each number from 1 to num
10    for i in range(1, num + 1):
11        factorial *= i
12
13    # Return the calculated factorial
14    return factorial
15
16 # Get input from user
17 n = int(input("Enter a number: "))
18
19 # Call function and display result
20 result = calculate_factorial(n)
21 print(f"Factorial of {n} is {result}")

```

The terminal window at the bottom shows the execution of the script:

```

PS C:\2403A51\03\3-2\AI_A_C> & 'c:\Users\isrch\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\isrch\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\libs\debugpy\launcher' '62973' '-...' 'c:\2403A51\03\3-2\AI_A_C\Lab1'
Enter a number: 6
Factorial of 6 is 720
PS C:\2403A51\03\3-2\AI_A_C>

```

The left sidebar shows the 'RUN AND DEBUG' panel with options to 'Run and Debug', 'To customize Run and Debug create a launch.json file.', 'Debug using a terminal command or in an interactive chat.', and 'Show automatic Python configurations'. The bottom status bar indicates 'Ln 21, Col 39', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', '3.12.3', 'Go Live', and 'Prettier'.

- **Modularity improves reusability by:**

Allowing the `calculate_factorial()` function to be reused in multiple programs without rewriting code.

Making the program easier to test, update, and debug.

Improving code organization, where logic is separated from input/output handling.

Supporting scalability, as the same function can be extended or integrated into larger projects.

Task Description

Compare the non-function and function-based Copilot-generated programs on the following criteria:

- Logic clarity
- Reusability
- Debugging ease
- Suitability for large projects
- AI dependency risk

The screenshot shows a Visual Studio Code editor window with a Python script for calculating factorials. The script is as follows:

```
13 def calculate_factorial(num):
22     # Return the calculated factorial
23     return factorial
24
25 # Get input from user
26 n = int(input("Enter a number: "))
27
28 # Call function and display result
29 result = calculate_factorial(n)
30 print(f"Factorial of {n} is {result}")
31
```

Below the code, a terminal window displays the execution output:

```
PS C:\2403A51L03\3-2\AI_A_C>
PS C:\2403A51L03\3-2\AI_A_C> & 'c:\Users\isrch\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\isrch\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62973' '--' 'C:\2403A51L03\3-2\AI_A_C\Lab1'
Enter a number: 6
Factorial of 6 is 720 ...
PS C:\2403A51L03\3-2\AI_A_C> c:: cd 'c:\2403A51L03\3-2\AI_A_C'; & 'c:\Users\isrch\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\isrch\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51942' '--' 'C:\2403A51L03\3-2\AI_A_C\Lab1'
Enter a number: 5
Factorial of 5 is 120
PS C:\2403A51L03\3-2\AI_A_C>
```

The interface also includes a 'RUN AND DEBUG' sidebar on the left with options like 'Run and Debug', 'To customize Run and Debug create a launch.json file.', 'Debug using a terminal command or in an interactive chat.', and 'Show automatic Python configurations'. The bottom status bar shows 'Ln 32, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', '3.12.3', 'Go Live', and 'Prettier'.


```

31
32 # Comparison of Non-Function vs Function-Based Factorial Programs
33
34 print("\n=== COMPARISON ===\n")
35
36 comparison = {
37     "Logic Clarity": {
38         "Non-Function": "Direct and simple, but logic is mixed with I/O",
39         "Function-Based": "Clearer separation of concerns; logic isolated in function"
40     },
41     "Reusability": {
42         "Non-Function": "Cannot reuse; must rewrite code for each use",
43         "Function-Based": "Highly reusable; call function multiple times"
44     },
45     "Debugging Ease": {
46         "Non-Function": "Harder; must debug entire script together",
47         "Function-Based": "Easier; test function independently in isolation"
48     },
49     "Large Projects": {
50         "Non-Function": "Not suitable; leads to code duplication and maintenance issues",
51         "Function-Based": "Suitable; promotes modularity and organization"
52     },
53     "AI Dependency Risk": {
54         "Non-Function": "Lower risk; simple, self-contained logic",
55         "Function-Based": "Higher risk if function relies on external AI; more dependencies"
56     }
57 }
58
59 for criterion, comparison_data in comparison.items():
60     print(f"{criterion}:")
61     for approach, description in comparison_data.items():
62         print(f"    {approach}: {description}")
63     print()

```

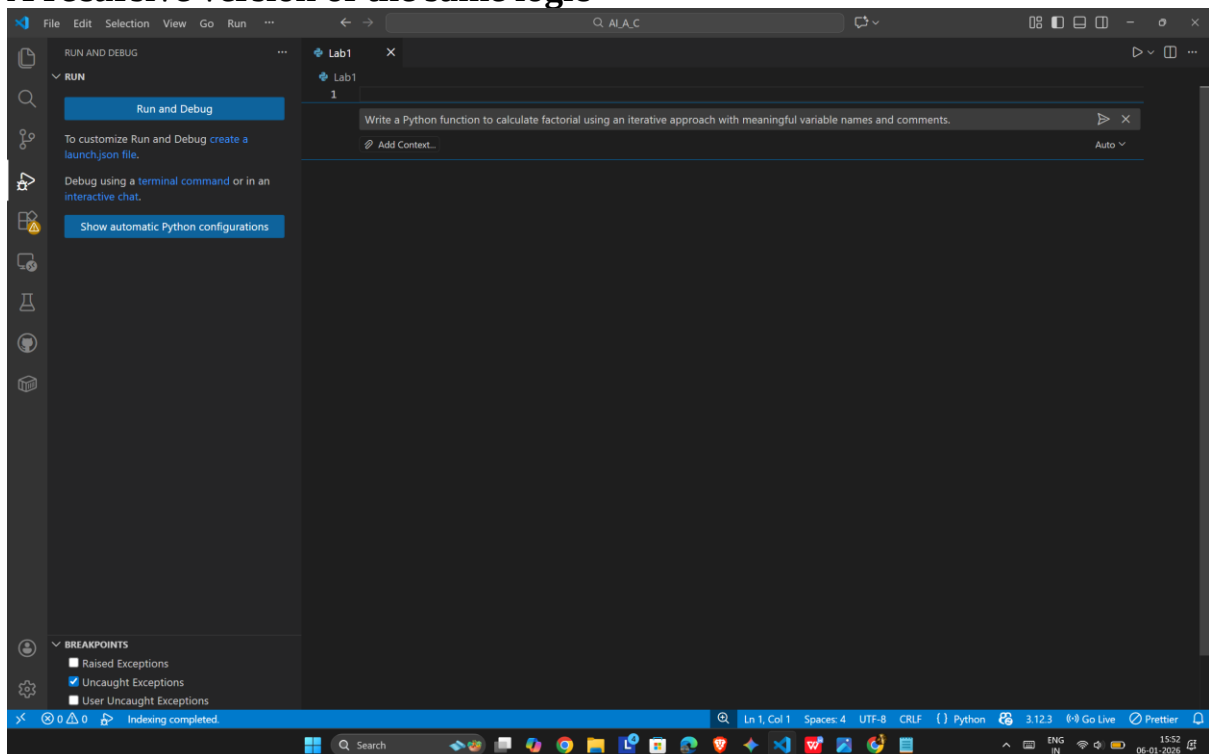
TASK - 5

Task Description

Prompt Copilot to generate:

An iterative version of the logic

A recursive version of the same logic



The screenshot shows the Visual Studio Code editor with a Python file named 'Lab1'. The code defines a function 'calculate_factorial(n)' that calculates the factorial of a non-negative integer using iteration. The function includes a docstring, input validation, a base case, and a loop to calculate the result. An example usage is provided at the bottom.

```

1 def calculate_factorial(n):
2     """
3     Calculate the factorial of a non-negative integer using iteration.
4
5     Args:
6         n: A non-negative integer
7
8     Returns:
9         The factorial of n
10    """
11    # Validate input
12    if n < 0:
13        raise ValueError("Factorial is not defined for negative numbers")
14
15    # Base case: factorial of 0 is 1
16    if n == 0 or n == 1:
17        return 1
18
19    # Initialize result to 1
20    factorial_result = 1
21
22    # Multiply result by each number from 2 to n
23    for current_number in range(2, n + 1):
24        factorial_result *= current_number
25
26    return factorial_result
27
28 # Example usage
29 if __name__ == "__main__":
30     test_value = 5
31     result = calculate_factorial(test_value)
32     print(f"Factorial of {test_value} is {result}")
33

```

The left sidebar shows the 'RUN AND DEBUG' panel with options to 'Run and Debug', 'To customize Run and Debug create a launch.json file.', 'Debug using a terminal command or in an interactive chat.', and 'Show automatic Python configurations'. The bottom status bar indicates 'Ln 33, Col 52', 'Spaces: 4', 'UTF-8', 'Python', '3.12.3', 'Go Live', and 'Prettier'.

The screenshot shows the Visual Studio Code editor with a Python file named 'Lab1'. The code defines a function 'calculate_factorial(n)' that calculates the factorial of a non-negative integer using recursion. The function includes a docstring, input validation, a base case, and a recursive call. An example usage is provided at the bottom.

```

18 def calculate_factorial(n):
19     """
20     Calculate the factorial of a non-negative integer using recursion.
21
22     Args:
23         n: A non-negative integer
24
25     Returns:
26         The factorial of n
27    """
28    # Validate input
29    if n < 0:
30        raise ValueError("Factorial is not defined for negative numbers")
31
32    # Base case: factorial of 0 is 1
33    if n == 0 or n == 1:
34        return 1
35
36    # Recursive case: factorial of n is n multiplied by factorial of n-1
37    return n * calculate_factorial(n - 1)
38
39 # Example usage
40 if __name__ == "__main__":
41     test_value = 5
42     result = calculate_factorial(test_value)
43     print(f"Factorial of {test_value} is {result}")
44

```

The left sidebar shows the 'RUN AND DEBUG' panel with options to 'Run and Debug', 'To customize Run and Debug create a launch.json file.', 'Debug using a terminal command or in an interactive chat.', and 'Show automatic Python configurations'. The bottom status bar indicates 'Ln 36, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', '3.12.3', 'Go Live', and 'Prettier'.

The screenshot shows the Visual Studio Code editor with a Python file named 'Lab1'. The code defines a recursive function 'calculate_factorial_recursive(n)' and uses it to calculate the factorial of 5. The function includes input validation, a base case, and a recursive case. The output is printed as 'Factorial of {test_value} is {result}'.

```

33 print(f"Factorial of {test_value} is {result}")
34
35
36 def calculate_factorial_recursive(n):
37     """
38     Calculate the factorial of a non-negative integer using recursion.
39
40     Args:
41         n: A non-negative integer
42
43     Returns:
44         The factorial of n
45     """
46     # Validate input
47     if n < 0:
48         raise ValueError("Factorial is not defined for negative numbers")
49
50     # Base case: factorial of 0 or 1 is 1
51     if n == 0 or n == 1:
52         return 1
53
54     # Recursive case: n! = n * (n-1)!
55     return n * calculate_factorial_recursive(n - 1)
56
57
58 # Example usage with recursive function
59 if __name__ == "__main__":
60     test_value = 5
61     result = calculate_factorial_recursive(test_value)
62     print(f"Factorial of {test_value} (recursive) is {result}")

```

OUTPUT:

The screenshot shows the Visual Studio Code editor with the same Python file. The output window at the bottom displays the execution results, showing the factorial of 5 is 120.

```

PS C:\2403A51103\3-2\AI_A_C> cd 'c:\2403A51103\3-2\AI_A_C'; & 'c:\Users\isrch\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\isrch\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61331' '-...' 'C:\2403A51103\3-2\AI_A_C\Lab1'
Factorial of 5 is 120
Factorial of 5 (recursive) is 120
PS C:\2403A51103\3-2\AI_A_C>

```

Explanation

- Iterative Approach

- **Starts with a result value of 1.**
- **Repeats multiplication from 1 up to the given number using a loop.**
- **Stores the intermediate result in the same variable.**
- **Executes sequentially without extra memory overhead.**

- Recursive Approach

- **Breaks the problem into smaller subproblems.**
- **Each function call multiplies the current number by the factorial of the previous number.**
- **Stops when it reaches the base case (0 or 1).**
- **Uses the call stack to remember previous function calls.**